BigQuery Architecture

Explained

How Google BigQuery is built: storage, compute, networking, query execution, ingestion, security and the design choices behind its speed.

Limits, quotas and pricing change over time. Treat numbers here as typical defaults and confirm them in the official BigQuery documentation before designing around them.

Contents

1. What BigQuery Is 3

2. High-Level Architecture 3

3. Core Building Blocks 5

4. Storage Architecture 5

5. Compute Architecture (Dremel and Slots) 6

6. Query Lifecycle 7

7. Data Ingestion and Consumption Paths 8

8. Resource Hierarchy and Locations 8

9. Security Architecture 8

10. Performance Features 9

11. Reliability and Availability 9

12. Related Components in the BigQuery Ecosystem 9

13. Design Best Practices (Architect's Checklist) 9

14. Interview-Ready Q&A; 10

1. What BigQuery Is

BigQuery is Google Cloud's fully managed, serverless **data warehouse** for analytics. You load data, write SQL, and Google handles servers, scaling, replication, patching and tuning. Under the hood it is a massively parallel processing (MPP) system that can scan terabytes in seconds and petabytes in minutes.

The four design ideas everything else follows from

Design idea	What it means	Why it matters to you
Separate storage and compute	Data lives in Colossus; queries run on a separate pool of workers called slots	Scale each independently; pay for each separately; no cluster to size or resize
Columnar storage	Each column is stored on its own, compressed	Queries read only the columns they name, so less I/O and less cost
Massively parallel execution	One query is split across thousands of slots	Speed comes from parallelism, not from one big machine
Serverless	No VMs, indexes or vacuum jobs to manage	You focus on SQL and data modeling instead of infrastructure

What BigQuery is not

- **Not an OLTP database.** It is built for large scans and aggregations, not for thousands of single-row lookups or updates per second.
- **Not index-driven.** There are no B-tree indexes. Speed comes from column pruning, partition pruning, clustering and parallel scans (plus optional search indexes for text lookups).

2. High-Level Architecture

Think of BigQuery as five cooperating layers. Requests come in at the top, the control plane decides how to run them, the compute layer does the work, and the storage layer holds the data. A very fast network joins compute to storage.

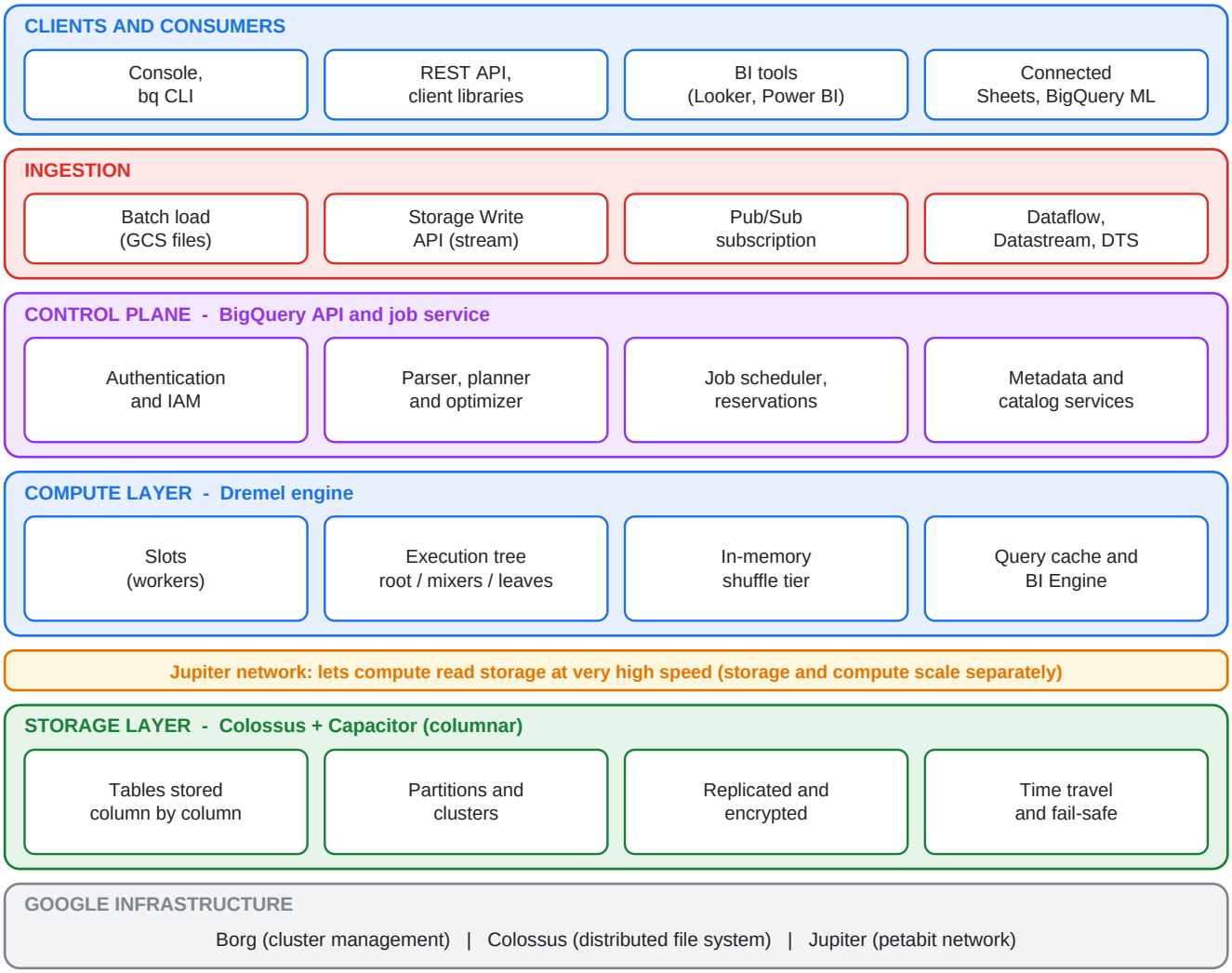


Figure 1: BigQuery layered architecture

Layer by layer

Layer	Role	Key technologies
Clients and consumers	Where people and tools submit SQL and read results	Console, bq CLI, REST API, client libraries, JDBC/ODBC, Looker, Power BI, Connected Sheets
Ingestion	Brings data in as batch loads, streams or change data capture	Load jobs, Storage Write API, Pub/Sub BigQuery subscriptions, Dataflow, Datastream, Data Transfer Service
Control plane	Authenticates, parses, optimizes and schedules every job; manages metadata	Job service, query optimizer, IAM, reservations and scheduler, metadata services
Compute	Runs query stages in parallel on slots, with shuffle between stages	Dremel engine, in-memory shuffle, BI Engine
Storage	Holds table data durably in columnar form	Colossus file system, Capacitor format, replication, encryption
Infrastructure	Provides machines, scheduling and the network	Borg, Jupiter, Colossus

3. Core Building Blocks

Component	What it is	Role in BigQuery
Dremel	Google's distributed SQL execution engine	Splits a query into stages and runs them on many slots using a multi-level execution tree
Colossus	Google's distributed file system (successor to GFS)	Stores all table data durably, replicated, and encrypted
Capacitor	BigQuery's columnar storage file format	Stores each column separately with heavy compression and encodings; reorders rows to compress better
Jupiter	Google's datacenter network	Petabit-scale bandwidth so compute can read remote storage as fast as local disk
Borg	Google's cluster manager (ancestor of Kubernetes)	Allocates the machines that run slots and services
Slot	A unit of compute: CPU, memory and I/O	The worker that executes one piece of one stage of a query
Shuffle tier	Distributed in-memory data exchange between stages	Moves intermediate data for joins, GROUP BY and ORDER BY without writing to disk

4. Storage Architecture

4.1 Columnar storage (Capacitor on Colossus)

A row store keeps all fields of one row together. BigQuery keeps all values of one **column** together. For analytics this is a large win, because queries usually touch a few columns out of many.

```
Table: orders(order_id, customer_id, country, amount, notes, ... 40 columns)

Row store     : [id|cust|country|amount|notes...] [id|cust|country|amount|notes...] ...
Column store  : [id,id,id,...] [cust,cust,...] [country,country,...] [amount,...] ...

SELECT country, SUM(amount) FROM orders GROUP BY country;
-> reads ONLY the country and amount column files.  The other 38 are never touched.
```

- **Compression and encoding:** similar values sit next to each other, so dictionary, run-length and other encodings work very well.
- **Immutable files:** data files are not edited in place. DML such as UPDATE or MERGE writes new files and swaps metadata, which is why BigQuery is better at bulk changes than many tiny ones.
- **Automatic storage optimization:** BigQuery compacts and reorganizes files in the background, and re-clusters clustered tables, at no extra query cost.
- **Durability:** data is encrypted at rest by default and replicated within the dataset location (multiple zones for regional datasets).

4.2 Partitioning and clustering

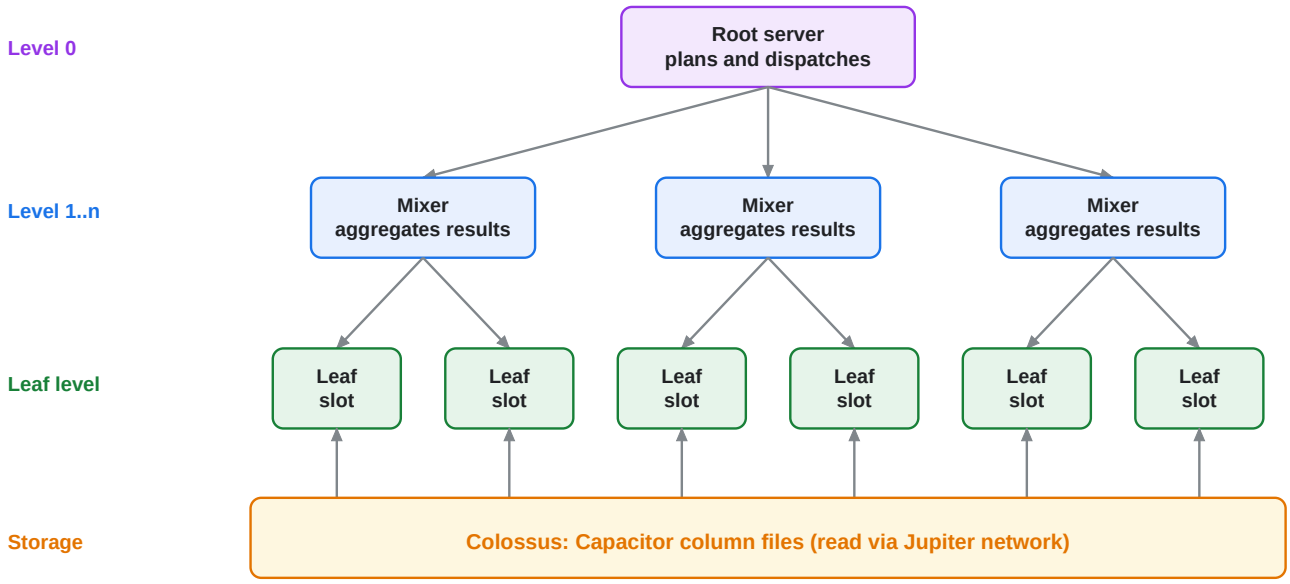
Technique	How it works	Benefit
Partitioning	Table is split into segments by a DATE/TIMESTAMP column, ingestion time, or an integer range	Queries that filter on the partition column skip whole partitions (partition pruning) and cost less
Clustering	Rows inside each partition are sorted by up to 4 chosen columns	Filters and joins on those columns skip blocks of data (block pruning)
Both together	Partition by date, cluster by high-cardinality filter columns	Typical best practice for large fact tables

4.3 Storage tiers, time travel and fail-safe

Concept	Detail
Active storage	Tables or partitions modified in the last 90 days
Long-term storage	Unmodified for 90 consecutive days; billed at a lower rate automatically, with no change in performance
Time travel	Query or restore data as of a point in the last 2 to 7 days (default 7), configurable per dataset
Fail-safe	An additional 7-day disaster-recovery window after time travel; not for routine use, recovery goes through Google support
Logical vs physical billing	Storage can be billed on uncompressed (logical) or compressed (physical) bytes, chosen per dataset. Highly compressible data can be cheaper on physical billing

5. Compute Architecture (Dremel and Slots)

When a query runs, Dremel builds an **execution tree**. The root receives the query and breaks it into pieces. Mixers in the middle combine partial results. Leaf nodes, which are slots, read data from storage and do the first round of filtering and aggregation. Partial results flow upward until the root returns the final answer.



Results flow upward (partial aggregates merged at each level). Data is read at the bottom.

Figure 2: Dremel execution tree (conceptual)

Modern BigQuery goes beyond a fixed tree. It runs the query as a **directed graph of stages** and uses a separate in-memory **shuffle tier** to repartition data between stages. The scheduler can add or remove slots **while the query runs** (dynamic execution), for example if one stage is skewed and needs more parallelism.

Slots

- A slot is BigQuery's unit of compute. Thousands of slots can work on one query at the same time.
- Slots are allocated per stage, so a query may use many slots for a big scan and fewer for the final merge.
- Under contention, slots are shared fairly across concurrent queries in the same project or reservation.

Pricing and capacity models

Model	How it works	Good for
On-demand	Pay per TiB of data processed by each query. Slots come from a shared pool with a default per-project cap	Spiky, ad-hoc or small workloads; getting started

Model	How it works	Good for
Capacity (editions)	Pay for slot-hours in Standard, Enterprise or Enterprise Plus editions. Autoscaling slots, optional 1 or 3-year commitments, and reservations to split slots among teams or workloads	Predictable cost, steady heavy workloads, workload isolation (ETL vs BI)

6. Query Lifecycle

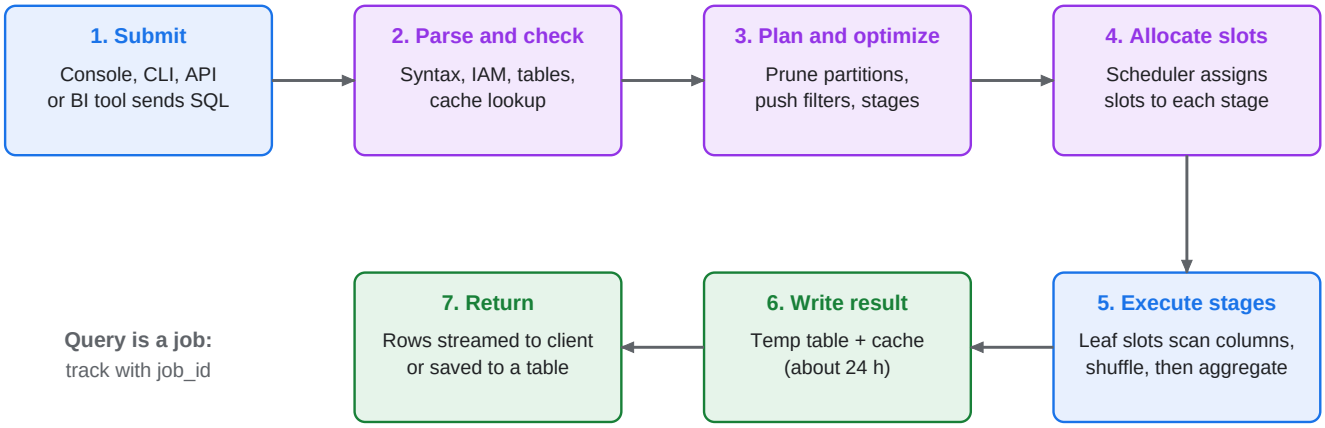


Figure 3: What happens when you run a query

Worked example

```
SELECT country, SUM(amount) AS revenue
FROM   sales.orders
WHERE  order_date BETWEEN '2026-01-01' AND '2026-01-31'
GROUP  BY country;
```

Stage	What happens
Plan	Optimizer sees the date filter and keeps only the January partitions. Only the order_date, country and amount columns are scheduled for reading
Stage 1: Scan	Many leaf slots each read a slice of the column files, apply the date filter, and compute a partial SUM per country
Stage 2: Shuffle	Partial results are hashed by country and sent through the shuffle tier so every row for one country lands on the same slot
Stage 3: Aggregate	Slots add up the partial sums for each country
Stage 4: Output	Final rows are gathered, written to a temporary result table (cached for about 24 hours) and returned to the client

Reading the execution details

- In the console, open a job and view **Execution details**. Each stage shows records read and written plus time spent in wait, read, compute and write.
- Big gaps between average and maximum compute time on one stage point to **data skew**, often a hot join or GROUP BY key.
- Large **shuffle** volumes usually mean an expensive join or a GROUP BY on a high-cardinality key; filter and aggregate earlier.
- **Query cache:** an identical query on unchanged tables returns from cache for free. Queries with non-deterministic functions such as CURRENT_TIMESTAMP() are not cached.

7. Data Ingestion and Consumption Paths

Path	Mechanism	Notes
Batch load	Load job from Cloud Storage or local file (CSV, JSON, Avro, Parquet, ORC)	Free of query charges; best for scheduled bulk loads
Streaming	Storage Write API (preferred) or legacy tabledata.insertAll	Rows are available for query within seconds; supports exactly-once semantics with the Write API
Pub/Sub	BigQuery subscription writes messages directly to a table	No Dataflow code needed for simple event ingestion
CDC	Datastream replicates changes from MySQL, PostgreSQL, Oracle, etc.	Keeps a warehouse copy of operational databases in near real time
Pipelines	Dataflow (Apache Beam), Dataproc (Spark), Data Fusion, Airflow/Composer, dbt	Transform before or after loading (ETL or ELT)
SaaS and cloud transfers	BigQuery Data Transfer Service	Scheduled imports from Google Ads, YouTube, Amazon S3 and others
External data	External tables and BigLake over Cloud Storage; federated queries to Cloud SQL, Spanner	Query data without loading it, with some performance trade-offs
Consumption	SQL, BI Engine, Looker, Power BI, Connected Sheets, BigQuery ML, Storage Read API	Storage Read API gives Spark and pandas fast parallel reads

8. Resource Hierarchy and Locations

```
Organization
  +-- Folder (optional grouping: team, environment)
        +-- Project            <- billing, quotas, IAM, jobs run here
              +-- Dataset      <- location (region / multi-region), access control
                    +-- Table, View, Materialized View
                    +-- Routine (UDF, stored procedure)
                    +-- Model (BigQuery ML)
```

- **Project** is the billing and quota boundary, and the place where query jobs execute.
- **Dataset** is the unit of location and the main access-control boundary. Its location cannot be changed after creation; you copy data to a new dataset instead.
- **Locations:** a single region (for example asia-south1 Mumbai) or a multi-region (US, EU). Queries must run in the same location as the data they read.
- **Cross-region:** joining datasets in different locations is not allowed directly; replicate or copy data first. Dataset replication and managed disaster recovery are available for resilience.

9. Security Architecture

Layer	Feature
Encryption	Encrypted at rest and in transit by default. Optional customer-managed keys (CMEK) through Cloud KMS
Identity and access	IAM roles at organization, project, dataset, table and view level. Service accounts for pipelines
Fine-grained access	Authorized views and routines, row-level security policies, column-level security with policy tags, dynamic data masking
Network perimeter	VPC Service Controls to reduce data exfiltration risk; Private Google Access
Governance	Data Catalog / Dataplex for lineage and discovery, DLP for sensitive data discovery
Audit	Cloud Audit Logs record admin activity and data access; INFORMATION_SCHEMA.JOBS records every query

10. Performance Features

Feature	What it does
Column pruning	Reads only the columns referenced; avoid SELECT *
Partition and block pruning	Skips partitions and clustered blocks that cannot match the filter
Query result cache	Free repeat results for identical queries on unchanged data
Materialized views	Precomputed, incrementally refreshed aggregates; the optimizer can rewrite queries to use them automatically
BI Engine	In-memory acceleration layer for dashboards, with sub-second response for common BI queries
Search index	Fast lookups for text and log search without full scans
Dynamic execution	Rebalances slots and repartitions data mid-query to handle skew

11. Reliability and Availability

- **Compute is stateless.** If a worker fails, the scheduler reruns that piece of the stage elsewhere; you never see it.
- **Storage is replicated** across zones in a region (and more widely for multi-region), so a zone outage does not lose data.
- **Recovery tools:** time travel, table snapshots and clones, and fail-safe cover accidental deletes and bad writes.
- **Resilience across regions:** cross-region dataset replication and managed disaster recovery are options for critical workloads.
- **Shared responsibility:** Google runs the platform; you still own data modeling, access policies, cost controls and backup strategy.

12. Related Components in the BigQuery Ecosystem

Component	Purpose
BigQuery ML	Create and run ML models (regression, classification, time series, forecasting, remote models) using SQL
BigLake	Unified storage engine for lakehouse data in open formats (Parquet, Iceberg) with BigQuery-style governance
BigQuery Omni	Run BigQuery analytics over data stored in AWS S3 or Azure Blob
Analytics Hub	Securely share and subscribe to datasets across organizations
Dataform / dbt	SQL-based transformation workflows with versioning and testing
Gemini in BigQuery	AI assistance for SQL generation, explanation and data preparation

13. Design Best Practices (Architect's Checklist)

- Partition large tables by date and cluster on common filter and join keys; require partition filters on the biggest tables.
- Choose columns deliberately; never SELECT * on wide tables.
- Use ELT: land raw data first, then transform with SQL inside BigQuery. Layer datasets as raw, staging, silver and gold.
- Prefer batch load jobs over streaming when freshness of a few minutes is acceptable.
- Use MERGE or partition overwrite for incremental loads; avoid many single-row DML statements.
- Separate workloads with reservations (ETL, BI, data science) and set per-job byte limits.
- Control access with dataset-level IAM, authorized views, and policy tags for sensitive columns.

- Monitor with INFORMATION_SCHEMA.JOBS, set budgets and alerts, and label jobs and resources for cost attribution.
- Keep datasets in the same region as their sources and consumers to avoid cross-region friction.

14. Interview-Ready Q&A;

Question	Short answer
How is BigQuery different from a traditional warehouse?	Serverless, with storage and compute separated; columnar; no indexes or cluster sizing; elastic parallelism through slots
What is Dremel?	The execution engine that splits a query into stages and runs them across thousands of slots using a tree and shuffle tier
Where is the data stored?	In Colossus, in the Capacitor columnar format, replicated and encrypted by default
What is a slot?	A unit of compute (CPU, memory, I/O) that executes part of a query stage
On-demand vs capacity pricing?	On-demand bills per bytes scanned; capacity bills for slot-hours with reservations and autoscaling
Why is BigQuery fast on huge scans?	Column pruning, partition and clustering pruning, massive parallelism, and a fast Jupiter network between compute and storage
Partitioning vs clustering?	Partitioning splits a table into segments for pruning at a coarse level; clustering sorts data within segments for fine-grained pruning
What are time travel and fail-safe?	Time travel lets you query or restore data from up to 7 days back; fail-safe is an extra 7-day recovery window handled by Google
How do you reduce query cost?	Select fewer columns, filter on partitions, cluster, use materialized views, dry-run queries, cap bytes billed, and consider capacity pricing